Relatório técnico — Compactador e descompactador

Turma: A1

Integrantes: Camillo Monaco Guides (11202320367), Carolina Luiza Deitos Simões da Camara e Silva (11202520449), Irene de Jesus Massareli (11202521587), Maisa Aragão Silva (11202111988)

Programa: compactador de arquivos de texto com códigos de Huffman por token

1. Objetivo e visão geral

O programa oferece, por meio de um menu no terminal, a compactação e a descompactação de arquivos. Na compactação, ele lê o arquivo para contar tokens, constrói uma árvore de Huffman, atribui códigos binários e grava um arquivo em formato próprio com extensão ‘.sccp’. Na descompactação, lê o dicionário e a árvore gravados no cabeçalho e reconstrói o conteúdo em um arquivo escolhido pelo usuário.

O método é Huffman aplicado a tokens: sequências de letras, dígitos e bytes UTF-8 são tratadas como tokens; espaços, pontuação e outros separadores também são tokens. Portanto, o programa não codifica cada palavra isoladamente e descarta a pontuação: os separadores são preservados como símbolos do dicionário.

2. Requisitos implementados

Requisitos	**Situação**
Menu com compactar, descompactar e sair	Implementado em [main.c]
Seleção de arquivo de entrada e destino	Implementada.
Extensão própria para compactação	Implementada. Como `.sccp`
Nome de destino escolhido para descompactação	Implementado. Como `.txt`.
Preservação do arquivo original	O programa abre a entrada para leitura e grava o resultado em outro caminho

Huffman e dicionário necessários para restaurar	Implementados e armazenados no arquivo compactado.
Tamanhos e razão de compactação	Exibidos após a compactação
Tempo de processamento	Exibido nas duas operações
Exibição opcional de frequência e código dos tokens	Implementada como opção na compactação.

3. Fundamentação conceitual

3.1 Frequência, dicionário e Tabela Hash

Na primeira leitura, cada token distinto é inserido no dicionário e sua frequência é incrementada nas ocorrências seguintes, sendo a definição de token, neste trabalho, uma palavra, um caractere especial que não uma letra (‘,’ ou ‘!’, por exemplo) ou separações (espaço ou “enter”, quebra de linha). A tabela hash associa cada token ao índice de sua entrada no dicionário, acelerando a busca média. A função hash usa a recorrência `hash = hash * 31 + byte, sendo o byte representado pela variável texto[i] (token i da entrada do dicionário), desde que seu valor seja diferente de ‘\0’, e também a multiplicidade por 31 pois é um número primo bastante utilizado para tabelas hash de strings. Após isso, os conflitos são tratados por encadeamento. Em outras palavras, é aplicada a função de espalhamento hash e, em seguida, é aplicado o módulo da divisão desta função pela capacidade em tamanho da tabela (return hash%capacidade). Depois é testado se o token pertence à um “bucket” ou nó que já esteja ocupado, se sim, encadeia o novo token ao seu equivalente em módulo pela capacidade.

Se ‘N’ é o número de bytes lidos, ‘U’ o número de tokens distintos e ‘L’ o comprimento médio do token, a varredura é aproximadamente linear em ‘N’, além do custo de procurar/inserir tokens. A busca hash tem custo médio proporcional a ‘L’, mas pode chegar a ‘O(U × L)’ em colisões desfavoráveis.

3.2 Árvore e códigos de Huffman

A árvore de Huffman é construída a partir das frequências dos tokens identificados na etapa anterior. O algoritmo seleciona repetidamente os dois nós de menor frequência e os combina em um novo nó, cuja frequência corresponde à soma dos dois. Esse processo continua até que reste apenas um nó, que representa a raiz da árvore. A partir dela, são atribuídos códigos binários aos tokens: cada caminho para a esquerda representa o bit 0, enquanto cada caminho para a direita representa o bit 1. Dessa forma, tokens mais frequentes tendem a receber códigos menores, reduzindo a quantidade total de bits necessária para representar o arquivo.

3.3 Gravação e decodificação dos bits

Os bits dos códigos são acumulados em bytes, do bit mais significativo para o menos significativo. O último byte é completado com zeros, e o cabeçalho registra a quantidade exata de bits válidos. Na descompactação, o leitor percorre a árvore bit a bit e escreve o token ao alcançar uma folha; o limite de bits impede que o preenchimento final seja interpretado como conteúdo.

O arquivo '.sccp' grava, em formato binário nativo da máquina:

1. quantidade de bits codificados ('unsigned long long');
2. quantidade de tokens do dicionário ('int');
3. tamanho em bytes de cada token;
4. árvore em pré-ordem, usando '0' para nó interno e '1' seguido do índice do token para folha;
5. fluxo de bits Huffman.

Como o cabeçalho usa representações nativas de inteiros e não inclui versão ou checksum, o formato não foi projetado para portabilidade garantida entre arquiteturas ou para detectar corrupção.

4. Bibliotecas e decisões de implementação

O programa foi escrito em C e não usa biblioteca externa de compressão. As principais bibliotecas são:

- 'stdio.h': arquivos e entrada/saída ('fopen', 'fread', 'fwrite', 'fgetc', 'printf');
- 'stdlib.h': alocação dinâmica e liberação de memória;

- ‘string.h’: operações com strings;

- ‘ctype.h’: incluída em [dicionario.c], embora não seja necessária no fluxo atual;

- ‘time.h’: medição por ‘clock()’;

- ‘windows.h’ e ‘commdlg.h’: diálogos nativos de abertura e salvamento no Windows.

Além das bibliotecas padrão, nós organizamos as estruturas e operações em módulos próprios (‘.h’ e ‘.c’). Esses cabeçalhos não são bibliotecas externas, e sim declaram os tipos e as funções implementados pelo próprio grupo e usados pelos demais arquivos do programa. Os arquivos ‘.h’ servem para auxiliar na compactação, enquanto os arquivos ‘.c’ são as implementações dos códigos.

Módulo	Responsabilidade
[dicionario.h] / [dicionario.c]	Define ‘Palavra’ e ‘Dicionario’; lê a entrada, separa e contabiliza tokens e gerencia o vocabulário
[hash.h] / [hash.c]	Define a tabela hash com encadeamento e implementa a função hash, inserção, busca e liberação
[huffman.h] / [huffman.c]	Define ‘NoHuffman’ e implementa a árvore de Huffman, a geração dos códigos e a liberação da árvore
[compactador.h] / [compactador.c]	Implementa a gravação do dicionário e da árvore, a codificação e o empacotamento dos bits no arquivo ‘.sccp’
[descompactar.h] / [descompactar.c]	Implementa a leitura do cabeçalho, a reconstrução da árvore e a decodificação dos bits para restaurar os tokens
[selecaoArquivo.h]/[selecaoArquivo.c]	Abstrai a seleção de caminhos e usa diálogos nativos no Windows e entrada pelo terminal nas demais plataformas

O [main.c] coordena esses módulos, apresentando o menu, chamando as rotinas de compactação ou descompactação e exibindo os resultados. Os arquivos ‘.h’ fornecem as interfaces entre os módulos, enquanto as implementações permanecem nos arquivos ‘.c’.

Enquanto o [Makefile] testa se o sistema operacional é do tipo Windows. Se sim, configura o GCC com ‘-Wall -Wextra -g’ e liga ‘comdlg32’ no Windows. Caso contrário, executa a compilação com o sistema padrão do Linux. É importante ressaltar que não foram utilizadas bibliotecas prontas de Huffman, compressão ou ordenação.

4.1 Trechos representativos

Contagem de frequência: ao encontrar um token já existente, sua frequência é incrementada; caso contrário, ele é inserido no dicionário e na tabela hash (dicionario.c).

```c
int posicao = buscarHash(h, texto, d);

if (posicao != -1) {
    d->palavras[posicao].frequencia++;
    return 1;
}
```

Seleção gulosa dos menores nós: a construção da árvore repete a busca dos dois nós de menor frequência (huffman.c).

```c
while (quantidade > 1) {
    menor1 = encontrarMenorNo(nos, quantidade);
    esquerda = nos[menor1];
    nos[menor1] = nos[quantidade - 1];
    quantidade--;

    menor2 = encontrarMenorNo(nos, quantidade);
    direita = nos[menor2];
    pai = combinarNos(esquerda, direita);
    nos[menor2] = pai;
}
```

Empacotamento dos códigos: cada caractere ‘0’ ou ‘1’ desloca o byte atual; o byte é gravado ao completar oito bits (compactador.c).

```c
*byteAtual = (*byteAtual << 1) | (codigo[i] - '0');
(*bitsNoByte)++;

if (*bitsNoByte == 8) {
    if (fputc(*byteAtual, saida) == EOF) return 0;
    *byteAtual = 0;
    *bitsNoByte = 0;
}
```

Decodificação: a cada folha alcançada, o token associado é escrito no arquivo de saída; a travessia reinicia na raiz (descompactar.c).

```c
if (atual->esquerda == NULL && atual->direita == NULL) {
    int indice = atual->indiceDicionario;
    const char *token = d.palavras[indice].texto;

        if (fwrite(token, 1, strlen(token), saida) != strlen(token))
        return 0;

    atual = raiz;
}
```

5. Casos de teste e capturas de tela

Para os testes de compactação e descompactação, foram gerados três arquivos com escrita em Latim, mais conhecidos por textos “Lorem Ipsum”, através do gerador online *lipsum*.

Caso 1 — texto com 2 parágrafos

Entrada: [texto_2paragrafos.txt]

Captura da compactação:

```
Arquivo original: C:\Users\carol\OneDrive\┴rea de Trabalho\UFABC\ProgramaþÒo Estruturada (PE)\trabalho PE\texto_2paragra
fos.txt Tamanho: 1305 Bytes
Arquivo comprimido: C:\Users\carol\OneDrive\┴rea de Trabalho\UFABC\ProgramaþÒo Estruturada (PE)\trabalho PE\branco.txt T
amanho: 1991 Bytes
Razao de compressao (R = Sc/So): 1.5257
Tempo de compressao: 0.002000 s
```

Captura da descompactação:

```
Arquivo comprimido: C:\Users\carol\OneDrive\┴rea de Trabalho\UFABC\ProgramaþÒo Estruturada (PE)\trabalho PE\branco.txt T
amanho: 1991 Bytes
Arquivo descompactado: C:\Users\carol\OneDrive\┴rea de Trabalho\UFABC\ProgramaþÒo Estruturada (PE)\trabalho PE\branco2.t
xt Tamanho: 1305 Bytes
Tempo de descompactacao: 0.003000 s
```

Resultado e análise: O arquivo compactado ficou 686 bytes maior que a entrada. A razão 1.991 / 1.305 = 1.5257 corresponde a um aumento de aproximadamente 52,57%. Para uma entrada curta, o custo do dicionário e da árvore armazenados no cabeçalho supera a economia obtida nos bits dos tokens. Este caso demonstra que Huffman não garante redução para todo tamanho ou distribuição de dados.

Caso 2 — texto com 25 parágrafos

-Entrada: [texto_25paragrafos.txt]

-Captura da compactação:

```
Arquivo original: C:\Users\carol\OneDrive\┴rea de Trabalho\UFABC\ProgramaþÒo Estruturada (PE)\trabalho PE\texto_25paragr
afos.txt Tamanho: 15577 Bytes
Arquivo comprimido: C:\Users\carol\OneDrive\┴rea de Trabalho\UFABC\ProgramaþÒo Estruturada (PE)\trabalho PE\branco.txt T
amanho: 6341 Bytes
Razao de compressao (R = Sc/So): 0.4071
Tempo de compressao: 0.004000 s
```

-Captura da descompactação:

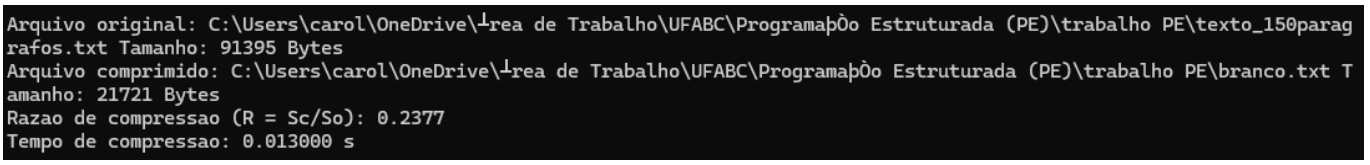
```
Arquivo comprimido: C:\Users\carol\OneDrive\┴rea de Trabalho\UFABC\ProgramaþÒo Estruturada (PE)\trabalho PE\branco.txt T
amanho: 6341 Bytes
Arquivo descompactado: C:\Users\carol\OneDrive\┴rea de Trabalho\UFABC\ProgramaþÒo Estruturada (PE)\trabalho PE\branco2.t
xt Tamanho: 15577 Bytes
Tempo de descompactacao: 0.035000 s
```

Resultado e análise: A razão `6.341 / 15.577 = 0,4071` indica redução de 59,29% (9.236 bytes). Neste arquivo, a repetição de tokens do texto permite que a codificação de Huffman compense o custo do cabeçalho. A descompactação preservou o tamanho original.

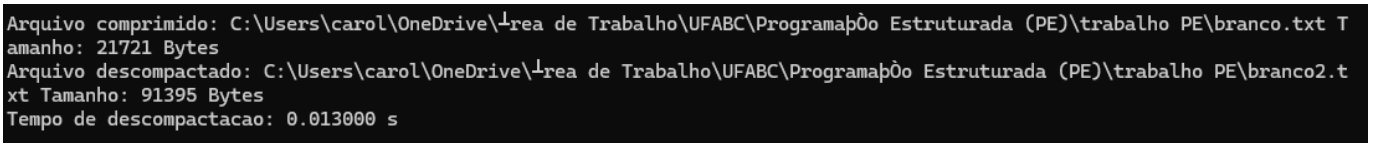
Caso 3 — texto com 150 parágrafos

Entrada: [texto_150paragrafos.txt]

Captura da compactação:



Captura da descompactação:



Resultado e análise: A razão `21.721 / 91.395 = 0,2377` representa redução de 76,23% (69.674 bytes), a maior entre os três casos. O arquivo maior amortiza o custo do cabeçalho e contém recorrências suficientes para que os tokens frequentes recebam códigos menores.

6. Medição e análise de desempenho

A razão de compactação usada pelo programa é:

$$R = \frac{S_c}{S_o}$$

Em que ‘Sc’ é o tamanho do arquivo compactado e ‘So’ é o tamanho original. ‘R < 1’ significa redução de espaço; ‘R = 1’, tamanho igual; e ‘R > 1’, expansão. A economia percentual, quando positiva, pode ser calculada por ‘(1 - R) × 100%’.

Em um dos casos medidos :

- So = 1305 B

- Sc = 1991 B

- R = 1991 / 1305 = 1.5257

- tempo de compactação exibido: 0.002000 s

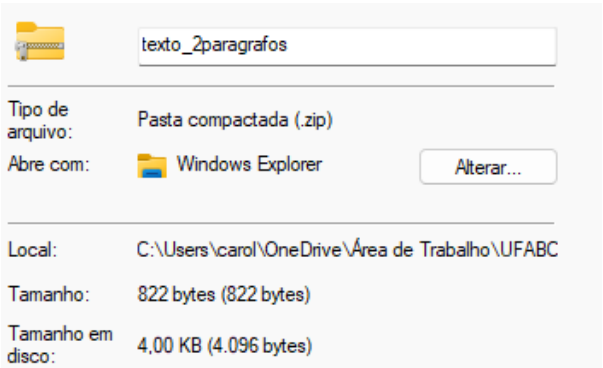
O arquivo ficou maior porque, nessa entrada curta e pouco redundante, o cabeçalho — dicionário, árvore e campos de controle — custa mais espaço do que os bits economizados pela

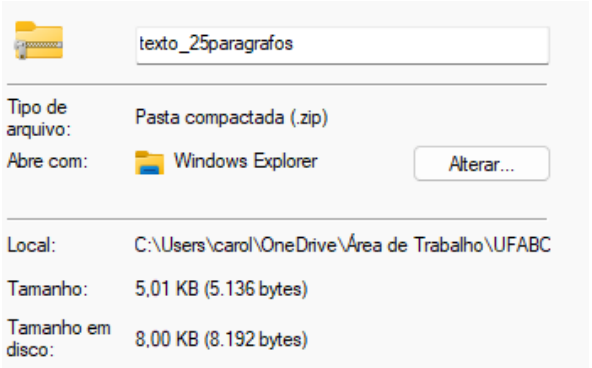
codificação. Isso é compatível com a natureza de Huffman: o ganho depende da distribuição de frequências, e os metadados têm custo fixo e variável.

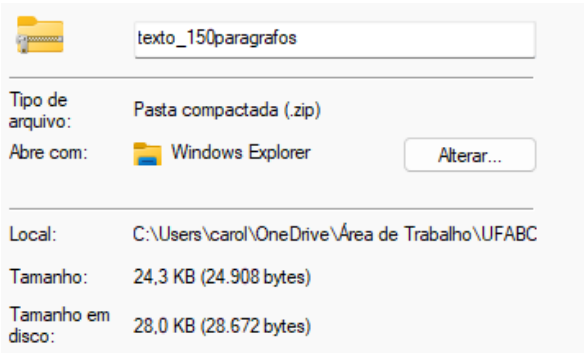
7. Comparação com ZIP do Windows

Registrou-se os tamanhos dos arquivos compactados pela opção ZIP do Windows, usada aqui como ferramenta equivalente para comparar o espaço ocupado. Os tamanhos dos arquivos '.zip' correspondentes foram conferidos e coincidem com os valores mostrados nas capturas. Além disso, o conteúdo de cada ZIP foi comparado por SHA-256 com a respectiva entrada e coincidiu.

Capturas das propriedades dos arquivos ZIP:







Entrada	Huffman do programa	ZIP do Windows	Razão do nosso programa	Razão ZIP	Comparação
texto_2 parágrafos.txt	1.991 B	822 B	1,5257	0,6299	ZIP é 1.169 B menor e reduz o arquivo em 37,01%. O nosso expande a entrada.

texto_25 parágrafos.txt	6.341 B	5.136 B	0,4071	0,3297	Ambos reduzem; ZIP é 1.205 B menor.
texto_150 parágrafos.txt	21.721 B	24.908 B	0,2377	0,2725	Ambos reduzem; o nosso é 3.187 B menor neste arquivo.

O ZIP produz o menor arquivo no caso de 2 e de 25 parágrafos, no caso de 150 parágrafos, o nosso fica menor por 3.187 bytes (12,80% do tamanho do ZIP). Essa comparação é apenas observacional, pois os formatos armazenam metadados diferentes. Para comparar também o desempenho temporal, ainda é necessário registrar os tempos, as versões e os comandos usados com a ferramenta escolhida.

8. Referências Bibliográficas

- BACKES, André. *Estrutura de dados descomplicada: em linguagem C*. Rio de Janeiro: Elsevier, 2016.

- BACKES, André. *Estrutura de Dados em C: Aula 90 — Tabela Hash — Implementação*. YouTube, 16 jun. 2015. Disponível em: https://www.youtube.com/watch?v=K40yG9bmVZ4. Acesso em: 3 nov. 2026.

- BACKES, André. *Estrutura de Dados em C: Aula 96 — Tabela Hash — Tratamento de Colisões*. YouTube, 27 jul. 2015. Disponível em: https://www.youtube.com/watch?v=wBReEzdR7So. Acesso em: 3 nov. 2026.

- CELES, Waldemar; CERQUEIRA, Renato; RANGEL, José. *Introdução a estruturas de dados: com técnicas de programação em C*. 2. ed. Rio de Janeiro: Elsevier, 2016.

- GABRIEL, Paulo Henrique Ribeiro. *Algoritmo de Huffman*. YouTube, 14 set. 2020. Disponível em: https://www.youtube.com/watch?v=BDwtGMhGNc0. Acesso em: 3 nov. 2026.

- LOREM IPSUM. *Lorem Ipsum – All the facts*. [S. l.], [s. d.]. Disponível em: https://www.lipsum.com/. Acesso em: 4 nov. 2026.

- SANTOS, Maria Carolina de Souza; DUARTE, Mauricio. Análise e implementação de algoritmos de compressão de dados. *Revista Eletrônica e-Fatec*, v. 2, n. 1, 2012. Disponível em: https://pesquisafatec.com.br/ojs/efatec/pt_BR/article/view/15. Acesso em: 3 nov. 2026.